



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2513 - TECNOLOGÍAS Y APLICACIONES WEB

Ejercicios I2

1º semestre 2026 - Profesor Hernán Cabrera

Ejercicio 1

Explique brevemente la estructura de un JWT y qué función cumple cada una de sus partes. Explique que son los rainbow tables (o el ataque rainbow tables) y como bcrypt lo previene.

Solución

Un JWT (*JSON Web Token*) está compuesto por tres partes separadas por puntos:

`header.payload.signature`

- **Header:** contiene información del token, como el algoritmo de firma utilizado (HS256, RS256, etc.) y el tipo de token (JWT).
- **Payload:** contiene los datos o *claims* del usuario, por ejemplo:

```
1 | { id: 123, role: "admin" }
```

- **Signature:** se genera firmando el header y payload utilizando una clave secreta. Permite verificar que el token no fue modificado.

Los **rainbow tables** son tablas precalculadas de hashes utilizadas para descubrir contraseñas comparando hashes conocidos con hashes almacenados.

bcrypt previene este ataque utilizando:

- **Salt aleatorio:** agrega datos aleatorios distintos a cada contraseña antes de hashearla.
- **Costo computacional:** bcrypt es intencionalmente lento, dificultando ataques masivos de fuerza bruta.

Ejercicio 2

Clasifique los códigos HTTP según su categoría (2xx, 3xx, 4xx, 5xx)

Solución

- **2xx:** respuestas exitosas.
 - 200 OK
 - 201 Created
- **3xx:** redirecciones.
 - 301 Moved Permanently
 - 302 Found
- **4xx:** errores del cliente.
 - 400 Bad Request
 - 401 Unauthorized
 - 404 Not Found
- **5xx:** errores del servidor.
 - 500 Internal Server Error
 - 503 Service Unavailable

Ejercicio 3

¿Cuál es la diferencia entre `Promise.all()` y `Promise.allSettled()`? ¿Cuándo usaría una en lugar de la otra?

Solución

`Promise.all()` espera que todas las promesas se resuelvan exitosamente. Si una sola promesa falla, toda la operación es rechazada inmediatamente.

```
1 | Promise.all([p1, p2, p3])
```

`Promise.allSettled()` espera que todas las promesas terminen, sin importar si fueron resueltas o rechazadas.

```
1 | Promise.allSettled([p1, p2, p3])
```

Diferencia principal:

- `Promise.all()` falla rápido ante un rechazo.
- `Promise.allSettled()` entrega el resultado individual de todas las promesas.

Uso típico:

-
- `Promise.all()` cuando todas las operaciones son necesarias.
 - `Promise.allSettled()` cuando interesa conocer el resultado de todas aunque algunas fallen.

Ejercicio 4

Describe el flujo (pasos secuenciales) completo que ocurre desde que un cliente hace una petición `POST /api/users` con un cuerpo en JSON hasta que el servidor responde con un `201 Created` y el usuario creado en JSON. Debe considerar las cabeceras HTTP relevantes, validación, uso de algún middleware de parseo, y el formato de la respuesta.

Solución

El cliente envía una petición HTTP:

`POST /api/users`

con headers como:

```
1 | Content-Type: application/json
2 | Accept: application/json
```

y un cuerpo JSON:

```
1 | {
2 |   "name": "Ana",
3 |   "email": "ana@mail.com"
4 | }
```

El servidor recibe la petición y un middleware como `koa-bodyparser` transforma el JSON en un objeto JavaScript accesible desde:

```
1 | ctx.request.body
```

Luego se validan los datos:

- campos obligatorios
- formato correcto
- reglas de negocio

Si la validación falla, se puede responder:

```
1 | ctx.status = 400;
```

Si los datos son válidos, el servidor crea el usuario en la base de datos y responde:

```
1 | ctx.status = 201;
2 | ctx.body = nuevoUsuario;
```

La respuesta HTTP tendrá:

```
1 | Content-Type: application/json
```

y el cuerpo contendrá el usuario creado en formato JSON.

Ejercicio 5

Explique que significa cada parte de este hash generado con bcrypt

`$2b$10$X5rT9kLmNpQrSuVwXyZ012.Yz8AbCdEfGhIjKlMnOpQrStUvWxYz`

Solución

`$2b$10$X5rT9kLmNpQrSuVwXyZ012.Yz8AbCdEfGhIjKlMnOpQrStUvWxYz`

- `$2b$`: versión del algoritmo bcrypt.
- `10$`: costo o número de rondas (*cost factor*). Mientras más alto, más lento es calcular el hash.
- `X5rT9kLmNpQrSuVwXyZ012`: salt aleatorio utilizado.
- El resto corresponde al hash generado para la contraseña junto al salt.

Ejercicio 6

```
1 | async function obtenerDatos() {
2 |   const response = await fetch('https://api.ejemplo.com/data');
3 |   const data = await response.json();
4 |   return data;
5 | }
6 |
7 | async function mostrar() {
8 |   const datos = obtenerDatos();
9 |   console.log(datos);
10 | }
11 |
12 | mostrar();
```

¿Qué imprime `console.log`? ¿Por qué y cómo se arregla?

Solución

`console.log(datos)` imprime una **Promise pendiente**:

```
1 | Promise { <pending> }
```

Esto ocurre porque:

```
1 | const datos = obtenerDatos();
```

retorna inmediatamente una Promise, ya que `obtenerDatos` es una función `async`.

Para obtener el valor real se debe usar `await`:

```
1 | async function mostrar() {  
2 |   const datos = await obtenerDatos();  
3 |   console.log(datos);  
4 | }
```

Ejercicio 7

```
1 | const authMiddleware = async (ctx, next) => {  
2 |  
3 |   const token = ctx.headers.authorization?.split(' ')[1];  
4 |  
5 |   if (!token) {  
6 |     ctx.status = 401;  
7 |     return;  
8 |   }  
9 |   try {  
10 |     const decoded = jwt.verify(token, secret);  
11 |     ctx.state.user = decoded;  
12 |   } catch (err) {  
13 |     ctx.status = 401;  
14 |     return;  
15 |   }  
16 |   next();  
17 | };  
18 |  
19 | app.use(authMiddleware);  
20 |  
21 | app.use(async ctx => {  
22 |   ctx.body = { user: ctx.state.user };  
23 | });
```

Hay un error en el flujo asincrónico. ¿Cuál es?

Solución

El error es que:

```
1 | next();
```

debería ser:

```
1 | await next();
```

En Koa los middlewares son asincrónicos y deben esperar correctamente la ejecución del siguiente middleware.

Si no se usa `await`, pueden ocurrir problemas en el flujo del onion model y el middleware podría terminar antes de tiempo.

Ejercicio 8

Un equipo de desarrollo está construyendo una aplicación web para un banco. Para implementar la funcionalidad de transferencia de fondos, un desarrollador propone crear el siguiente endpoint y utilizarlo desde el browser del cliente:

```
GET http://banco.cl/api/transferir?origen=123&destino=456&monto=50000
```

El desarrollador argumenta que, dado que TCP/IP asegura que los paquetes lleguen correctamente sin pérdida de datos, esta implementación es robusta y suficiente.

Explique por qué el uso del método HTTP propuesto es incorrecto desde el punto de vista arquitectónico y semántico.

Identifique el error conceptual del desarrollador respecto a las responsabilidades las capas del modelo TCP/IP versus la capa de aplicación HTTP.

Proponga una estructura correcta para esta operación.

Solución

El método HTTP propuesto es incorrecto porque una transferencia bancaria modifica el estado del servidor y no debería realizarse mediante GET.

Las peticiones GET deben ser:

- seguras (*safe*)
- sin efectos secundarios
- utilizadas para obtener información

Usar GET para transferencias puede generar problemas graves:

- caché accidental
- ejecución involuntaria al refrescar
- almacenamiento en historial del navegador
- exposición de datos sensibles en la URL

El desarrollador confunde responsabilidades de capas:

- TCP/IP asegura transporte confiable de paquetes.
- HTTP define la semántica de la operación de aplicación.

Aunque TCP entregue correctamente los datos, eso no significa que el método HTTP utilizado sea correcto.

Una estructura adecuada sería:

```
1 | POST /api/transferencias
```

con cuerpo JSON:

```
1 | {  
2 |   "origen": 123,  
3 |   "destino": 456,  
4 |   "monto": 50000  
5 | }
```

Ejercicio 9

Prediga la salida completa del siguiente programa. Justifique brevemente cada resultado marcado con `// ?`. E indique el orden en que se mostrarán en consola.

```
1 | const obj = {  
2 |   valor: 100,  
3 |   obtener: function() { return this.valor; },  
4 |   obtenerFlecha: () => this.valor  
5 | };  
6 |  
7 | console.log(obj.obtener());           // (a) ?  
8 | console.log(obj.obtenerFlecha());     // (b) ?  
9 |  
10 | setTimeout(() => console.log("inicio"), 0); // (c) ?  
11 | console.log("fin");                   // (d) ?
```

Solución

(a)

```
1 | console.log(obj.obtener());
```

imprime:

```
1 | 100
```

porque `this` hace referencia al objeto `obj`.

(b)

```
1 | console.log(obj.obtenerFlecha());
```

imprime:

```
1 | undefined
```

porque las funciones flecha no tienen su propio `this`; heredan el `this` del contexto exterior.

(c) y (d)

El orden de impresión es:

```
1 | 100
2 | undefined
3 | fin
4 | inicio
```

`setTimeout(..., 0)` no se ejecuta inmediatamente. El callback se envía a la Task Queue y se ejecuta cuando el Call Stack queda vacío.